

Laboratório de Programação 2

PROF. DOMINGOS BERNARDO



EMENTA

01 Apresentação da Disciplina

02 Método de Avaliação

03 Ementa da Disciplina

PROF. DOMINGOS BERNARDO





APRESENTAÇÃO DO PROFESSOR

- Nome, formação, experiência na área
- Motivo pelo qual escolheu a disciplina de laboratório de programação 2

PROF. DOMINGOS BERNARDO





PERGUNTAS AOS DISCENTES

- o que define um bom desenvolvedor de software?
- por que organizar o código é importante?
- qual é a principal vantagem da Programação Orientada a Objetos?





CRITÉRIOS DE AVALIAÇÃO

Em cada bimestre, o discente será avaliado por meio de:

- Prova teórica (P) – avalia a compreensão dos conceitos, fundamentos, princípios de programação orientada a objetos, arquitetura de software e padrões de projeto;
- Atividade prática (AP) – consiste no desenvolvimento de atividades ou etapas do projeto prático da disciplina, avaliando a aplicação dos conceitos estudados.

A nota bimestral (NB) será calculada pela média aritmética simples entre a prova e a atividade prática:



O QUE É LABORATÓRIO DE PROGRAMAÇÃO 2

- Construir software estruturado e orientado a objetos;
- Desenvolver aplicações em camadas;
- Aplicar padrões de projeto;
- Trabalhar com persistência em banco de dados;
- Desenvolver aplicações Web orientadas a objetos;
- Programar de forma profissional e organizada.



EMENTA

- Construção de software;
- Programação Orientada a Objetos;
- Arquitetura de software;
- Padrões de projeto;
- Coleções e refatoração de código;
- Persistência de dados;
- Desenvolvimento orientado a projetos.

A disciplina é mão na massa, com projeto prático.



OBJETIVO GERAL

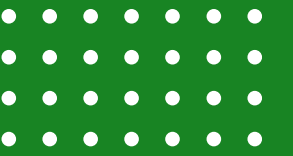
Compreender e aplicar os conceitos de Programação Orientada a Objetos no desenvolvimento de projetos de software, utilizando arquitetura em camadas, padrões de projeto e persistência de dados.





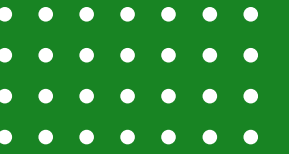
OBJETIVOS ESPECÍFICOS

- Desenvolver sistemas usando OO;
- Criar aplicações organizadas em camadas;
- Aplicar padrões de projeto;
- Trabalhar com banco de dados;
- Desenvolver aplicações Web;
- Refatorar e organizar código.



METODOLOGIA

- Aulas práticas em laboratório;
- Exercícios guiados;
- Projeto prático incremental;
- Discussão de código;
- Boas práticas de desenvolvimento.



AVALIAÇÃO BIMESTRAL

- Prova Teórica
- Projeto Prático
 - Evolução do sistema
 - Organização do código
 - Aplicação correta dos conceitos

MÓDULO 1 – FUNDAMENTOS DA LINGUAGEM

- Conteúdo: Fundamentos
 - Variáveis e constantes;
 - Tipos de dados;
 - Operadores;
 - Entrada e saída de dados.
- Estruturas de Controle
 - Condicionais (if / else);
 - Seleção múltipla (switch);
 - Laços de repetição (for, while).

MÓDULO 1 – FUNDAMENTOS DA LINGUAGEM

- Estruturas de Dados
 - Vetores;
 - Matrizes;
 - Coleções;
 - List, Set, Map.
- Recursos da Linguagem
 - Tratamento de exceções;
 - Manipulação de arquivos (I/O).



MÓDULO 2 – PROGRAMAÇÃO ORIENTADA A OBJETOS

- Introdução à OO
 - O que é Programação Orientada a Objetos?
 - Por que usar OO?
 - Comparação com programação procedural.
- Classes e Objetos
 - Classe;
 - Objeto;
 - Atributos;
 - Métodos;
 - Instanciação.



MÓDULO 2 – PROGRAMAÇÃO ORIENTADA A OBJETOS

- Encapsulamento
 - Modificadores de acesso
 - public, private, protected
 - Getters e setters
- Métodos
 - Construtores
 - Métodos estáticos
 - Métodos abstratos
 - Constantes



MÓDULO 2 – PROGRAMAÇÃO ORIENTADA A OBJETOS

- Herança
 - Superclasse e Subclasse;
 - Generalização;
 - Especialização.
- Polimorfismo
 - Sobrescrita de métodos;
 - Referência de superclasse;
 - Comportamento dinâmico.



MÓDULO 2 – PROGRAMAÇÃO ORIENTADA A OBJETOS

- Interfaces
 - Contratos;
 - Implementação;
 - Diferença entre classe abstrata e interface.



MÓDULO 3 – PROGRAMAÇÃO WEB

- Programação Web
 - O que é uma aplicação Web?
 - Cliente x Servidor
 - Arquitetura Web
- Back-end
 - Linguagens back-end
 - Frameworks
 - Organização do código



MÓDULO 3 – PROGRAMAÇÃO WEB

- Servidores Web
 - O que é um servidor Web?
 - Deploy de aplicações;
 - Ambientes de desenvolvimento.
- Interfaces Web
 - Camada de apresentação;
 - Separação entre lógica e interface.



MÓDULO 4 – PADRÕES DE PROJETO

- O que são Padrões de Projeto
 - Problemas recorrentes;
 - Soluções reutilizáveis;
 - Boas práticas.
- MVC
 - Model;
 - View;
 - Controller;
 - Benefícios do MVC.
 -



MÓDULO 4 – PADRÕES DE PROJETO

- Controle
 - Acoplamento;
 - Responsabilidade;
 - Manutenção do sistema.
- Persistência de Dados
 - Banco de dados;
 - Camada de persistência;
 - Separação de responsabilidades.



MÓDULO 4 – PADRÕES DE PROJETO

- O que são Padrões de Projeto
 - Problemas recorrentes;
 - Soluções reutilizáveis;
 - Boas práticas.
- MVC
 - Model;
 - View;
 - Controller;
 - Benefícios do MVC.
 -



MÓDULO 4 – PADRÕES DE PROJETO

- ORM
 - O que é ORM?
 - Mapeamento objeto-relacional;
 - Vantagens.



EXERCÍCIOS - RESPONDA

1. O que é um código compilado?
2. O que é um código interpretado?
3. Qual é a principal diferença entre um código compilado e um código interpretado?
4. Na sua opinião, em que tipo de situação é melhor usar um código compilado e em que situação é melhor usar um código interpretado?

The background is a dark blue gradient. It features several abstract, glowing particle trails in shades of light blue and white. These trails are composed of many small, bright dots connected by thin, curved lines, giving the impression of a nebula or a complex network of data points. The trails are scattered across the frame, with a larger, more dense cluster in the upper right and several smaller, more linear trails extending towards the bottom and left. The word "Obrigado" is centered in the middle of the image in a large, bold, white sans-serif font.

Obrigado

Laboratório de Programação 2

PROF. DOMINGOS BERNARDO



EMENTA

- 01 Variáveis e constantes
- 02 Tipos de dados
- 03 Operadores
- 04 Entrada e saída de dados

PROF. DOMINGOS BERNARDO





CONTEÚDO: FUNDAMENTOS

O que é uma Variável?

- Uma variável é um espaço na memória do computador usado para armazenar um valor.
- Pense como uma caixa que guarda informações, Exemplos: idade, nome, salário, temperatura.

Uma variável possui: nome, tipo, valor:

- `int idade = 18;`
- `double altura = 1.75;`
- `char letra = 'A';`
- `String nome = "Carlos";`
- `boolean aprovado = true;`



REGRAS PARA NOMES DE VARIÁVEIS

- ✓ Devem começar com letra;
- ✓ Podem conter números;
- ✓ Não podem ter espaços;
- ✓ Não usar palavras reservadas;
- ✓ O nome da variável deve ser em letras minúsculas;

correto:

nome

idadeAluno

total2

incorreto:

2nome

idade aluno

class



O QUE É UMA CONSTANTE?

Uma constante é um valor que não pode ser alterado durante o programa.

Constante em Java

Usamos a palavra **final**

```
final double PI = 3.14159;
```

```
final int DIAS_SEMANA = 7;
```



QUANDO USAR CONSTANTES?

Use constantes para valores que nunca mudam:

- ✓ PI
- ✓ dias da semana
- ✓ limite de velocidade
- ✓ taxa fixa
- ✓ valor do salário mínimo

- ✓ VALOR_MAXIMO
- ✓ TAXA_JUROS
- ✓ PI

Constantes são escritas em:

- LETRAS MAIÚSCULAS
- separadas por underline



A FUNÇÃO PRINCIPAL MAIN

Onde o programa começa?

Em Java, todo programa começa pela função:

```
public static void main(String[] args)
```

- 👉 É o ponto de entrada do programa.
- 👉 Sem ela, o programa não executa.
- 💡 Pense nela como a “porta de entrada” da aplicação.



A FUNÇÃO PRINCIPAL MAIN

```
public static void main(String[] args)
```

- ✓ public → acessível de qualquer lugar
- ✓ static → pode ser executado sem criar objeto
- ✓ void → não retorna valor
- ✓ main → nome padrão reconhecido pela JVM
- ✓ String[] args → permite receber argumentos do sistema



A FUNÇÃO PRINCIPAL MAIN

```
public class PrimeiroPrograma {  
  
    public static void main(String[] args) {  
  
        System.out.println("Olá, mundo!");  
  
    }  
  
}
```




O QUE É UMA CLASSE?

Uma classe é o modelo (estrutura) que organiza um programa Java. Pense como um molde que define como o programa será estruturado.

Em Java, todo código precisa estar dentro de uma classe.

```
public class PrimeiroPrograma {  
  
  
}
```



POR QUE O MAIN FICA DENTRO DA CLASSE?

Em Java:

- ✓ Tudo deve estar dentro de uma classe
 - ✓ A execução começa pelo método main
 - ✓ O main precisa estar dentro de uma classe para ser executado
- 👉 A JVM procura o método main dentro de uma classe para iniciar o programa.



ESTRUTURA BÁSICA DE UM PROGRAMA JAVA

Estrutura:

- ✓ Classe → organiza o programa
- ✓ main → ponto de início da execução
- ✓ instruções → o que o programa faz
- 💡 Todo programa Java segue essa estrutura básica.

```
public class PrimeiroPrograma {  
  
    public static void main(String[] args) {  
  
        System.out.println("Olá, mundo!");  
  
    }  
  
}
```



O QUE É A JVM?

JVM = Java Virtual Machine

A JVM é o programa responsável por executar aplicações Java.

👉 Ela funciona como um “computador virtual”.

Funções da JVM:

- ✅ Executar programas Java
- ✅ Interpretar o bytecode
- ✅ Gerenciar memória
- ✅ Garantir portabilidade

💡 Graças à JVM:

Java roda em qualquer sistema operacional.



COMO A JVM EXECUTA O PROGRAMA?

Etapas:

O código Java é escrito pelo programador

O compilador gera o bytecode (.class)

A JVM executa o bytecode

Fluxo:

 Código Java →  Compilação →  Bytecode →  JVM →  Execução



O QUE É O COMANDO JAVAC?

javac é o compilador Java.

➡ Ele traduz o código fonte (.java)

➡ para bytecode (.class)

Arquivo criado:

❏ Código

Programa.java



Comando:

❏ Código

javac Programa.java



Resultado:

❏ Código

Programa.class





O QUE É O COMANDO JAVA?

O comando java executa o programa.

👉 Ele chama a JVM para rodar o bytecode.

🔗 código

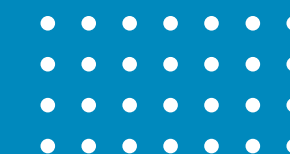


```
java Programa
```

🔗 Plain text



```
Programa.java
├
└─ javac
    └─
        Programa.class
            └─
                java
                    └─
                        JVM executa
```

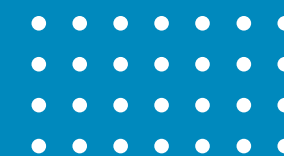


PRIMEIRO PROGRAMA JAVA

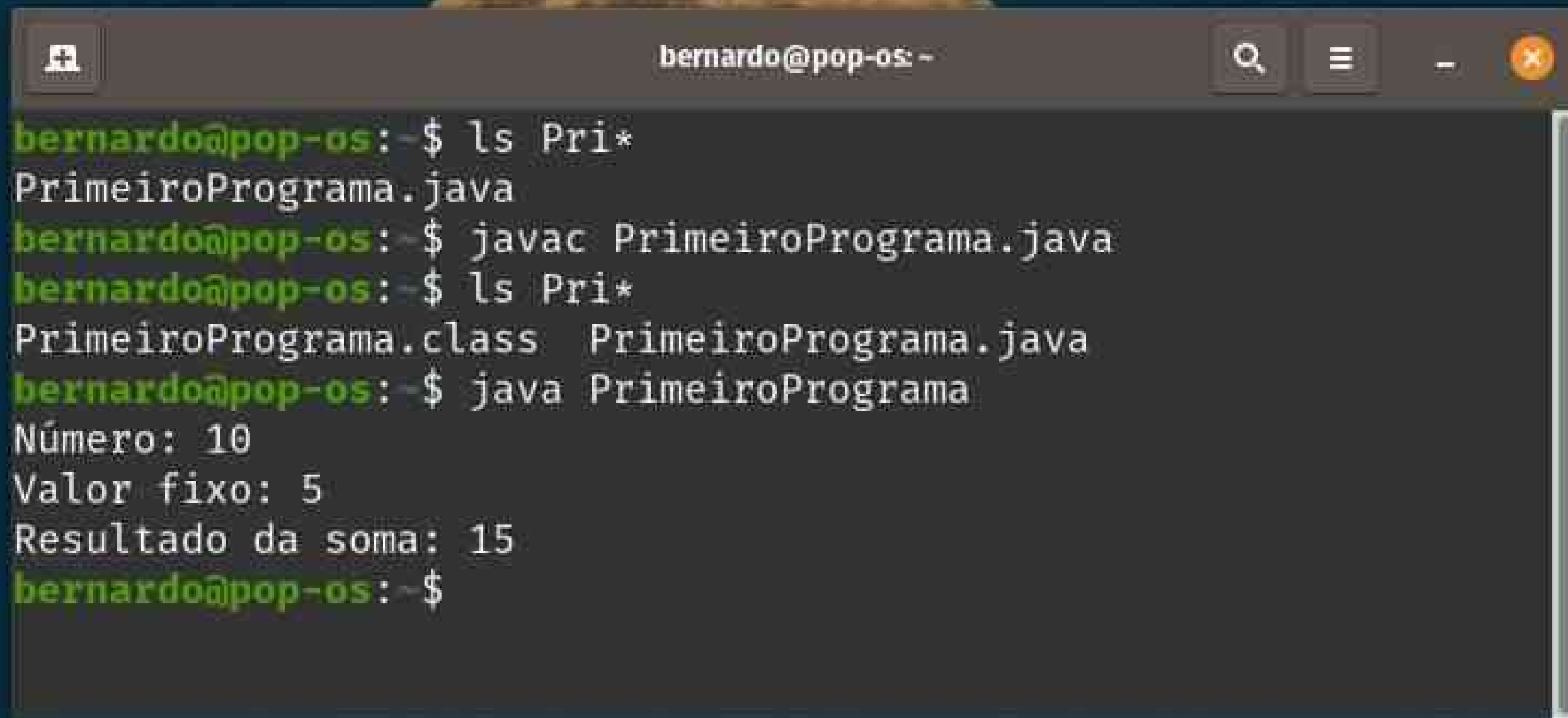
 Java



```
public class PrimeiroPrograma {  
  
    public static void main(String[] args) {  
  
        // variável  
        int numero = 10;  
  
        // constante  
        final int VALOR_FIXO = 5;  
  
        // cálculo  
        int resultado = numero + VALOR_FIXO;  
  
        // saída de dados  
        System.out.println("Número: " + numero);  
        System.out.println("Valor fixo: " + VALOR_FIXO);  
        System.out.println("Resultado da soma: " + resultado);  
  
    }  
}
```



COMPILAR E EXECUTAR CÓDIGO JAVA



```
bernardo@pop-os: ~  
bernardo@pop-os:~$ ls Pri*  
PrimeiroPrograma.java  
bernardo@pop-os:~$ javac PrimeiroPrograma.java  
bernardo@pop-os:~$ ls Pri*  
PrimeiroPrograma.class PrimeiroPrograma.java  
bernardo@pop-os:~$ java PrimeiroPrograma  
Número: 10  
Valor fixo: 5  
Resultado da soma: 15  
bernardo@pop-os:~$
```



O QUE SÃO TIPOS DE DADOS?

Os tipos de dados definem que tipo de valor uma variável pode armazenar.

- 👉 número
- 👉 texto
- 👉 verdadeiro ou falso
- 👉 um único caractere

💡 Eles ajudam o computador a reservar memória corretamente.



TIPOS PRIMITIVOS

Inteiros

Tipo	Tamanho	Exemplo
byte	8 bits	100
short	16 bits	2000
int	32 bits	10
long	64 bits	1000000L

Decimais

Tipo	Precisão
float	menor
double	maior



O QUE SÃO TIPOS DE DADOS?

Caractere

- Armazena um único caractere.
- **char letra = 'A';**

Booleano

- Armazena verdadeiro ou falso.
- **boolean aprovado = true;**

String (texto)

- Armazena palavras ou frases.
- **String nome = "Domingos";**



EXEMPLO COMPLETO

```
public class TiposDados {  
    public static void main(String[] args) {  
  
        int idade = 30;  
        double altura = 1.80;  
        char inicial = 'D';  
        boolean surfista = true;  
        String cidade = "Peruibe";  
  
        System.out.println("Idade: " + idade);  
        System.out.println("Altura: " + altura);  
        System.out.println("Inicial: " + inicial);  
        System.out.println("Surfista: " + surfista);  
        System.out.println("Cidade: " + cidade);  
    }  
}
```



ENTRADA E SAÍDA DE DADOS EM JAVA

Entrada (input)

Quando o programa recebe dados do usuário.

- 👉 teclado
- 👉 arquivos
- 👉 internet

Saída (output)

Quando o programa mostra informações.

- 👉 tela
- 👉 impressora
- 👉 arquivos



SAÍDA DE DADOS

System.out.println()

Exibe mensagem e pula linha.

System.out.print()

Exibe mensagem sem pular linha.

System.out.printf() (opcional)

Permite formatar a saída.

Java

```
System.out.println("Olá, mundo!");  
System.out.println("Java é legal!");
```

Código

```
Olá, mundo!  
Java é legal!
```

Java

```
System.out.print("Olá ");  
System.out.print("Domingos!");
```

Código

```
Olá Domingos!
```

Java

```
double altura = 1.75;  
System.out.printf("Altura: %.2f", altura);
```



ENTRADA DE DADOS COM SCANNER

Para ler dados do teclado usamos a classe:

Scanner

Antes de usar, precisamos importar:

```
❏ Java  
  
import java.util.Scanner;
```



Criando o Scanner

conecta o programa ao teclado.

```
❏ Java  
  
Scanner sc = new Scanner(System.in);
```





ENTRADA DE DADOS COM SCANNER

Lendo dados do usuário:

Texto (String)

```
// Java  
String nome = sc.nextLine();
```



Número inteiro

```
// Java  
int idade = sc.nextInt();
```



Número decimal

```
// Java  
double altura = sc.nextDouble();
```





EXEMPLO COMPLETO

```
import java.util.Scanner;

public class EntradaSaida {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Digite seu nome: ");
        String nome = sc.nextLine();

        System.out.print("Digite sua idade: ");
        int idade = sc.nextInt();

        System.out.print("Digite sua altura: ");
        double altura = sc.nextDouble();

        System.out.println("\n=== Dados Informados ===");
        System.out.println("Nome: " + nome);
        System.out.println("Idade: " + idade);
        System.out.println("Altura: " + altura);

        sc.close();
    }
}
```




EXERCÍCIOS

The background is a dark blue gradient. It features a complex network of thin, light blue lines that crisscross the frame. Some lines are straight, while others are curved or wavy. Scattered throughout the background are numerous small, bright blue and white particles, some of which form larger, denser clusters. The overall effect is one of dynamic energy and digital connectivity.

Obrigado

Laboratório de Programação 2

PROF. DOMINGOS BERNARDO

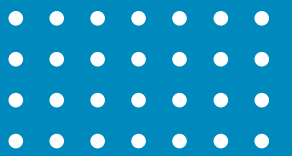


EMENTA

- 01 Classes e Objetos
- 02 Tipos Wrappers
- 03 Conversão entre tipos
- 04 InstanceOf e método equals

PROF. DOMINGOS BERNARDO





A IDEIA DE OBJETO EM JAVA

Java é uma linguagem orientada a objetos. Isso significa que quase tudo é tratado como objeto.

Um objeto possui:

- **Estado** → atributos (variáveis internas)
- **Comportamento** → métodos
- **Identidade** → referência na memória

Exemplo simples:

```
Java
String nome = "Domingos";
```

- String é uma classe;
- "Domingos" é um objeto;
- nome é uma referência para esse objeto.





TIPOS PRIMITIVOS VS OBJETOS

Tipos Primitivos:

- int;
- double;
- boolean;
- char;
- byte;
- short;
- long;
- float.

Características:

- Guardam o valor diretamente;
- Não possuem métodos;
- Não podem ser **null**.





TIPOS PRIMITIVOS VS OBJETOS

Objetos:

- String;
- Arrays;
- Classes criadas pelo programador;
- Wrappers (Integer, Double, etc.)

Características:

- Guardam referência para um objeto;
- Possuem métodos;
- Podem ser **null**.

```
// Java
```

```
String nome = "Domingos";
```





O QUE SÃO WRAPPERS?

No Java, nem tudo é um objeto por padrão. Temos os tipos primitivos (**eficientes, mas limitados**). Os **Wrappers** são classes que "embrulham" esses tipos primitivos, transformando-os em objetos.

Por que usar?

- Para usar em Coleções (como ArrayList), que só aceitam objetos.
- Para utilizar métodos utilitários (ex: Integer.parseInt("10")).
- Autoboxing/Unboxing: O Java faz a conversão automática entre eles.



Em Java, temos o sistema de tipos dividido em dois mundos:

1. **Tipos Primitivos:** Guardam valores diretamente na memória **Stack** (ex: int, double). São rápidos, mas não possuem métodos.
2. **Wrappers:** São classes que "envelopam" esses tipos primitivos, transformando-os em objetos que vivem na memória **Heap**.

Tipo Primitivo	Classe Wrapper	Exemplo de Uso
byte	Byte	Byte.MAX_VALUE
short	Short	shortValue()
int	Integer	Integer.parseInt("10")
long	Long	Long.sum(1, 2)
float	Float	Float.isNaN(f)
double	Double	Double.compare(d1, d2)
char	Character	Character.isDigit('1')
boolean	Boolean	Boolean.parseBoolean("true")



O QUE SÃO WRAPPERS?

Métodos Utilitários

Os Wrappers oferecem métodos prontos para conversão e manipulação de dados que os primitivos não têm.

```
1
2 public class Wrappers {
3
4     public static void main(String[] args) {
5
6         String texto = "123";
7
8         int numero = Integer.parseInt(texto); // Converte String para int
9
10        String binario = Integer.toString(10); // "1010"
11
12    }
13
14 }
15
```



O QUE SÃO WRAPPERS?

```
1
2 public class ExemploWrappers {
3     public static void main(String[] args) {
4         // Cenário: Recebemos dados de uma interface (sempre como String)
5         String precoTexto = "150";
6         String codigoAcesso = "A9";
7
8         // 1. Convertendo String para int usando o Wrapper Integer
9         int preco = Integer.parseInt(precoTexto);
10        System.out.println("Preço com taxa de conveniência (10%): " + (preco * 1.10));
11
12        // 2. Usando o Wrapper Character para validar se o segundo caractere é um número
13        char segundoChar = codigoAcesso.charAt(1);
14
15        if (Character.isDigit(segundoChar)) {
16            System.out.println("O caractere '" + segundoChar + "' é um número válido para o código.");
17        } else {
18            System.out.println("Erro: O segundo caractere deve ser um número.");
19        }
20
21        // 3. Verificando limites técnicos do tipo Inteiro
22        System.out.println("O maior valor que um int suporta é: " + Integer.MAX_VALUE);
23        System.out.println("O menor valor que um int suporta é: " + Integer.MIN_VALUE);
24    }
25 }
```




O QUE SÃO WRAPPERS?

```
1
2 public class MetodosWrappers {
3     public static void main(String[] args) {
4         // 1. Diferentes formas de instanciar (Autoboxing vs Construtor)
5         Integer pontuacao = 100; // Autoboxing (Recomendado)
6         Double precoBase = Double.valueOf("45.90"); // Método valueOf: cria objeto a partir de String
7
8         // 2. Usando métodos de instância (o objeto 'sabe' fazer coisas)
9         System.out.println("Preço em formato int: " + precoBase.intValue()); // 45
10        System.out.println("O preço é um número infinito? " + precoBase.isInfinite()); // false
11
12        // 3. Métodos Estáticos (Utilitários da Classe)
13        String valorVindoDoArquivo = "1250.75";
14
15        // Converter String diretamente para primitivo double
16        double valorConvertido = Double.parseDouble(valorVindoDoArquivo);
17
18        // Comparar dois valores usando o Wrapper
19        int resultadoComparacao = Integer.compare(pontuacao, 200);
20        // Retorna < 0 se o primeiro for menor, 0 se igual, > 0 se maior
21
22        if (resultadoComparacao < 0) {
23            System.out.println("A pontuação " + pontuacao + " é menor que 200.");
24        }
25
26        // 4. Verificando o tipo da instância com o método getClass()
27        // (Wrappers são objetos, então herdam de Object)
28        System.out.println("O objeto precoBase pertence à classe: " + precoBase.getClass().getSimpleName());
29    }
30 }
```



MÉTODOS DO WRAPPERS

Os métodos das classes Wrapper são ferramentas que o Java oferece para facilitar a vida do desenvolvedor ao lidar com números e caracteres. Como os tipos primitivos (int, double, etc.) são apenas valores brutos, o Java criou essas classes para "dar inteligência" a eles.

Podemos dividir esses métodos em três categorias principais:

1. Métodos de Conversão (Parsing)

São usados para transformar uma String (texto) em um valor numérico. É o uso mais comum em sistemas que recebem dados de formulários ou arquivos.

- **Integer.parseInt("10")** → Retorna um int.
- **Double.parseDouble("15.50")** → Retorna um double.



MÉTODOS DO WRAPPERS

2. Métodos de Instância (Extração)

Uma vez que você tem um objeto Wrapper, você pode querer o valor dele em outro formato.

- `objDouble.intValue()` → Pega um Double e extrai apenas a parte inteira.
- `objInteger.byteValue()` → Converte o valor para o tipo byte.

3. Métodos Utilitários (Estáticos)

São métodos que você chama direto na Classe para realizar verificações ou comparações rápidas.

- **`Character.isDigit('5')`** → Verifica se o caractere é um número.
- **`Character.isLetter('A')`** → Verifica se é uma letra.
- **`Integer.max(10, 20)`** → Retorna o maior entre dois valores.
- **`Integer.toBinaryString(10)`** → Transforma o número 10 em binário ("1010").



CONVERSÕES ENTRE WRAPPERS

A conversão entre Wrappers (ou entre Primitivos e Wrappers) segue regras bem específicas no Java. É importante mostrar que, embora o Java ajude muito com o **Autoboxing**, existem momentos em que precisamos ser explícitos.

Autoboxing é o processo automático pelo qual o Java converte um tipo primitivo em seu Wrapper correspondente.

Ou seja, o compilador **transforma** o valor **primitivo** em um **objeto** sem que o programador precise fazer isso manualmente.

Integer numero = 10; // int → Integer automaticamente



CONVERSÃO DE TIPOS

1. Autoboxing e Unboxing (A conversão "mágica")

Desde o Java 5, você não precisa mais "instanciar" um Wrapper manualmente. O Java faz a ponte entre o primitivo e o objeto de forma automática.

- Autoboxing: Primitivo -> Wrapper.

```
Java
Integer numero = 10; // O Java faz: Integer.valueOf(10)
```

- Unboxing: Wrapper → Primitivo.

```
Java
int valor = numero; // O Java faz: numero.intValue()
```



CONVERSÃO DE TIPOS WRAPPERS

2. Conversão entre tipos diferentes (Casting de Wrappers)

Atenção aqui: Diferente dos primitivos, você não pode fazer casting direto entre Wrappers de tipos diferentes (ex: de Integer para Double).

Java

```
Integer nota = 100;
```

```
// Forma 1: Usando o método de extração (mais elegante)
```

```
Double notaDouble = nota.doubleValue();
```

```
// Forma 2: Através do primitivo (unboxing + casting + autoboxing)
```

```
double temp = (double) nota;
```

```
Double notaDouble2 = temp;
```



CONVERSÃO DE TIPOS WRAPPERS

3. Conversão de String para Wrapper

Essa é a conversão mais usada na prática (ex: ler o que o usuário digitou).

Existem duas formas principais:

Método	Retorno	Uso Sugerido
<code>Integer.parseInt(s)</code>	<code>int</code> (primitivo)	Quando você só quer o valor para cálculos.
<code>Integer.valueOf(s)</code>	<code>Integer</code> (objeto)	Quando você vai guardar em uma Lista (<code>ArrayList</code>).



CONVERSÃO DE TIPOS WRAPPERS

3. Conversão de String para Wrapper

Essa é a conversão mais usada na prática (ex: ler o que o usuário digitou).

Existem duas formas principais:

Método	Retorno	Uso Sugerido
<code>Integer.parseInt(s)</code>	<code>int</code> (primitivo)	Quando você só quer o valor para cálculos.
<code>Integer.valueOf(s)</code>	<code>Integer</code> (objeto)	Quando você vai guardar em uma Lista (<code>ArrayList</code>).



CONVERSÃO DE TIPOS WRAPPERS

```
1
2 public class ConversaoWrappers {
3     public static void main(String[] args) {
4         // String para Wrapper
5         Integer idade = Integer.valueOf("20");
6
7         // Wrapper para outro tipo de primitivo
8         double idadeDouble = idade.doubleValue();
9         float idadeFloat = idade.floatValue();
10
11        // Wrapper para String
12        String texto = idade.toString();
13
14        System.out.println("Integer: " + idade);
15        System.out.println("Convertido para Double: " + idadeDouble);
16        System.out.println("Convertido para String: " + texto);
17    }
18 }
19
```



3 ERROS CLÁSSICOS COM WRAPPERS E CONVERSÕES

1. O Erro da String Inválida (NumberFormatException)

```
1
2 public class Erro {
3
4     public static void main(String[] args) {
5
6         String valor = "10.50";
7         // ERRO: parseInt espera apenas dígitos, não pontos ou vírgulas.
8         int preco = Integer.parseInt(valor);
9
10        String entrada = "R$ 100";
11        // ERRO: O símbolo "R$" impede a conversão.
12        int numero = Integer.parseInt(entrada);
13
14    }
15
16 }
17
```



3 ERROS CLÁSSICOS COM WRAPPERS E CONVERSÕES

2. Comparar Objetos com ==

Como Wrappers são objetos, o operador == compara o endereço de memória e não o conteúdo.

```
1
2 public class Erro {
3
4     public static void main(String[] args) {
5
6         Integer n1 = 200;
7         Integer n2 = 200;
8
9         if (n1 == n2) {
10             // Isso raramente será verdadeiro para números altos!
11         }
12
13         // ✅ O CORRETO:
14         if (n1.equals(n2)) {
15             System.out.println("Valores iguais!");
16         }
17     }
18 }
19
20 }
```



3 ERROS CLÁSSICOS COM WRAPPERS E CONVERSÕES

3. NullPointerException no Unboxing

Ao contrário dos tipos primitivos, um Wrapper pode ser null. Tentar transformar um null em um tipo primitivo causa o crash do programa.

```
1
2 public class Erro {
3
4     public static void main(String[] args) {
5
6         Integer pontuacao = null;
7
8         // ERRO: O Java tenta fazer pontuacao.intValue() e o objeto é nulo!
9         int total = pontuacao + 10;
10
11     }
12
13 }
```



== VS EQUALS() EM WRAPPERS

Quando trabalhamos com tipos primitivos, o **operador ==** compara diretamente o valor.

```
Java  
int a = 10;  
int b = 10;  
  
System.out.println(a == b); // true
```

Mas com Wrappers é diferente

Wrappers são objetos.

E objetos são comparados por referência quando usamos ==.

```
Java  
Integer a = 200;  
Integer b = 200;  
  
System.out.println(a == b); // false  
System.out.println(a.equals(b)); // true
```




== VS EQUALS() EM WRAPPERS

Por que isso acontece?

- == compara se os dois objetos estão no mesmo endereço de memória
- equals() compara o conteúdo (valor interno)

Ou seja:

- == → compara identidade
- equals() → compara valor

Essa diferença existe porque:

- Primitivos armazenam valor direto
- Objetos armazenam referência
- Java é orientado a objetos
- Wrappers são objetos



INSTANCEOF

Verificação de Tipo em Tempo de Execução

Em Java, o operador instanceof é utilizado para verificar se um objeto pertence a determinada classe ou implementa determinada interface.

Ele responde à pergunta:

“Este objeto é uma instância deste tipo?”

```
1
2 public class TesteInstance {
3
4     public static void main(String[] args) {
5
6         Object obj = "Java";
7
8         if (obj instanceof String) {
9             System.out.println("É uma String");
10        }
11
12    }
13
14 }
```



EXEMPLO BÁSICO COM INTEGER

- 10 é primitivo;
- Ao ser colocado em Object, ocorre autoboxing;
- O tipo real passa a ser Integer;
- Por isso instanceof Integer retorna true.

```
1
2 public class TesteInstance {
3
4     public static void main(String[] args) {
5
6         Object obj = 10; // autoboxing → Integer
7
8         if (obj instanceof Integer) {
9             System.out.println("É um Integer!");
10        }
11
12    }
13
14 }
```



EXEMPLO BÁSICO COM DOUBLE

- Mesmo sendo número, Double $\neq$ Integer
- Cada wrapper é uma classe diferente

```
1
2 public class TesteInstance {
3
4     public static void main(String[] args) {
5
6         Object obj = 3.14; // autoboxing → Double
7
8         if (obj instanceof Double) {
9             System.out.println("É um Double!");
10        }
11
12        if (obj instanceof Integer) {
13            System.out.println("É um Integer!");
14        }
15
16    }
17
18 }
```



INSTANCEOF COM NULL

- instanceof nunca lança erro com null
- Apenas retorna false
- Isso é importante para segurança.

```
1
2 public class TesteInstance {
3
4     public static void main(String[] args) {
5
6         Integer numero = null;
7
8         System.out.println(numero instanceof Integer); // false
9
10    }
11
12 }
13
```



EXERCÍCIOS

The background is a dark green field filled with numerous thin, light green lines that resemble particle tracks or fiber-optic paths. These lines crisscross the frame in various directions. A prominent feature is a dense, glowing cluster of green particles located in the upper-middle section of the image. Other smaller, less dense clusters of particles are visible in the lower-left and lower-right areas. The overall effect is one of dynamic energy and complex network connectivity.

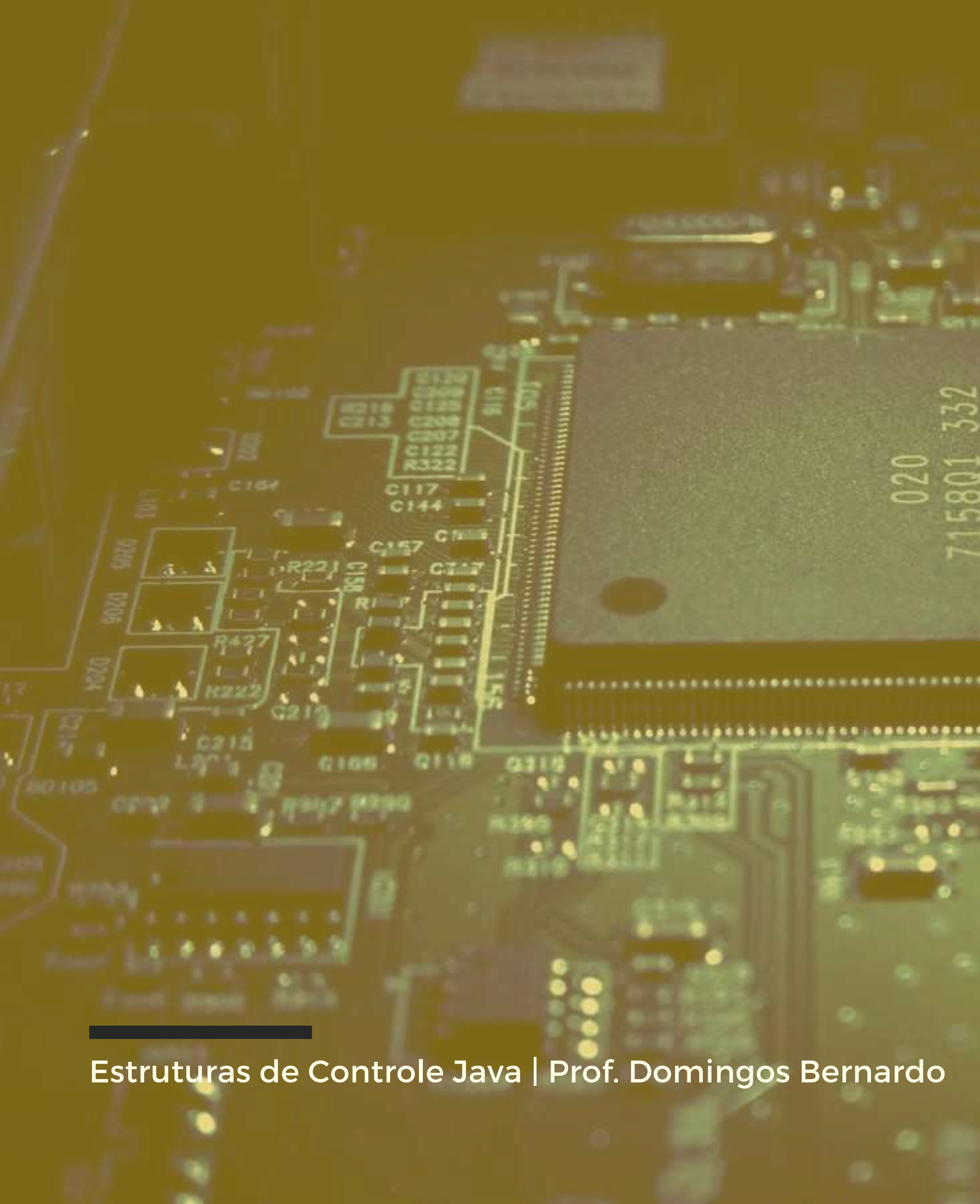
Obrigado

The background of the slide is a dark, semi-transparent overlay on a photograph of electronic hardware. On the left, there is a bundle of white, blue, and red cables. The central and right portions show a green printed circuit board (PCB) populated with various electronic components, including several electrolytic capacitors of different sizes and integrated circuits. The overall aesthetic is technical and modern.

LINGUAGEM DE PROGRAMAÇÃO

Estruturas de Controle Java

PROF. DOMINGOS BERNARDO



Tópicos do vídeo

- conceito sobre estruturas de controle Java;
- comandos de tomada de decisão: if else, e switch/case;
- comandos de repetição: while, do/while e for;
- Referências Bibliográficas.

Estruturas de Controle

- A linguagem de programação Java inclui suporte a diversas instruções que são utilizadas para controlar a estrutura lógica e de fluxo das aplicações.
- As instruções de estrutura de controle tem papel fundamental no desenvolvimento de aplicações Java, quebram o fluxo de execução, empregando tomada de decisão, loop e ramificação, permitindo que seu programa execute condicionalmente blocos de código específicos.





IF / ELSE

- A instrução **if** sempre deve receber uma condição para avaliar, que retorna o valor verdadeiro ou falso (true/false);
- Caso a condição retorne o valor igual a **true** (verdadeiro) o bloco do **if** é executado;
- Caso a condição retorne o valor igual a **false** (falso) o bloco do **else** é executado.



Tomada de decisão

if else

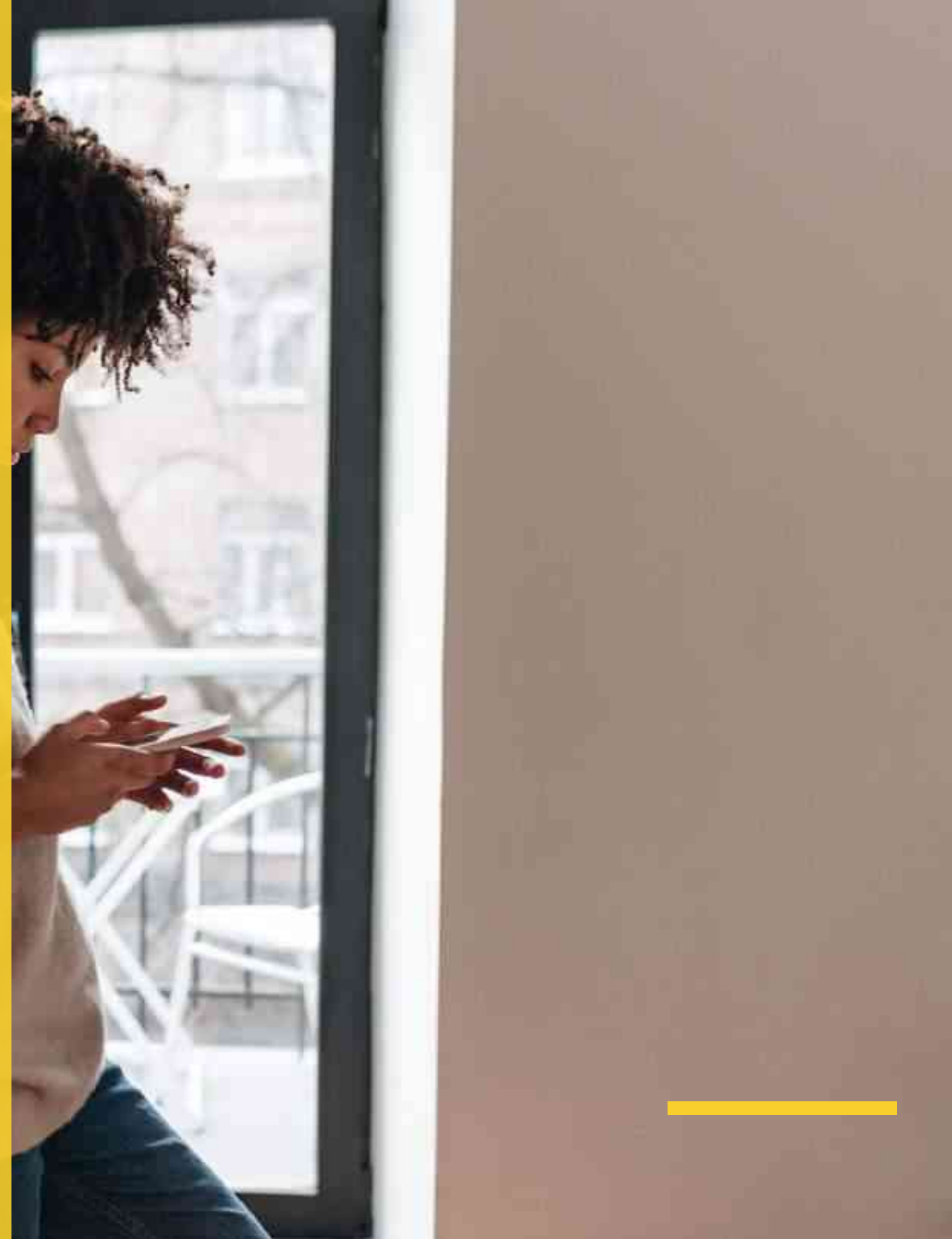
```
public class ExemploIf {  
    public static void main(String[] args) {  
        int x = 10;  
        if( x == 10 )  
            System.out.println("Valor de x igual a 10");  
        else  
            System.out.println("Valor de x diferente de 10");  
    }  
}
```

Primeiro foi definido o valor 10 para variável x do tipo inteiro int. Em seguida, o comando if verifica se a variável x é igual a 10. Caso a variável x seja igual a 10 o programa deve imprimir a mensagem "Valor de x igual a 10", caso contrário o programa deve imprimir a mensagem "Valor de x diferente de 10".



SWITCH / CASE

- Ao contrário das instruções if-else a instrução **switch** pode ter vários caminhos de execução possíveis.
- A instrução **switch** funciona com os tipos de dados primitivos byte, short, char e int;
- Também pode ser utilizado com String e classes Wrappers: Character, Byte, Short e Integer.



Tomada de decisão

switch case

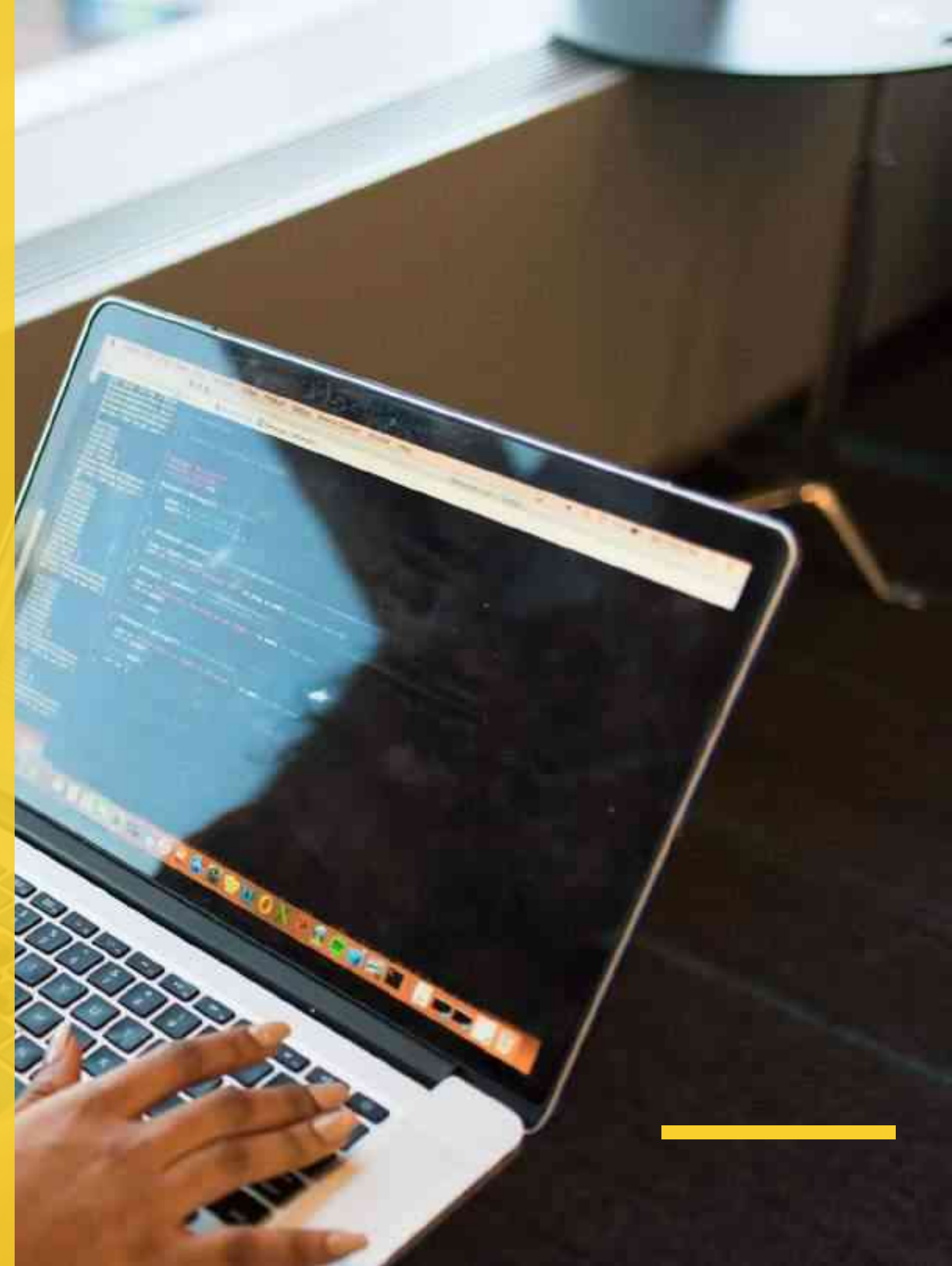
```
public class ExemploSwitch {  
    public static void main(String[] args) {  
        int numero = 3;  
        switch (numero) {  
            case 1: System.out.println("um");  
                    break;  
            case 2: System.out.println("dois");  
                    break;  
            case 3: System.out.println("três");  
                    break;  
            default: System.out.println("outro numero");  
                    break;  
        }  
    }  
}
```

Primeiro foi definido o valor 3 para variável numero do tipo inteiro int. Em seguida, a instrução switch verifica o valor da variável numero. Caso a variável numero seja igual a 1 o programa deve imprimir "um", Caso a variável numero seja igual a 2 o programa deve imprimir "dois". Caso o switch não reconheça o valor então o comando switch executa a opção default enviando a mensagem "outro numero" ao usuário."



WHILE

- Através da instrução **while**, é possível definir quantas vezes um determinado trecho de código deve ser executado pelo computador;
- A instrução **while** deve receber uma condição para avaliar a repetição;
- O processo de repetição deve continuar enquanto a condição definida no **while** estiver verdadeira.



Comando de Repetição

while

```
public class While1 {  
    public static void main(String[] args) {  
        int x = 0;  
        while (x < 10) {  
            System.out.println(x);  
            x = x + 1;  
        }  
    }  
}
```

Primeiro foi definido o valor zero para variável x do tipo inteiro int. Em seguida, o comando while verifica se a variável x é menor que 10. Caso a variável x seja menor que 10 o programa deve imprimir o valor de x. Toda vez que imprimir o valor de x, o programa deve incrementar o valor de x (somar 1). Quando o valor de x for igual a 10 o programa para de imprimir e encerrar a execução.



DO WHILE

- Através das instruções **do while**, é possível definir quantas vezes um determinado trecho de código deve ser executado pelo computador;
- A instrução **while** deve receber uma condição para avaliar a repetição;
- O processo de repetição deve continuar enquanto a condição definida na instrução **while** estiver verdadeira.



Comando de Repetição

do while

```
public class DoWhile1 {  
    public static void main(String[] args) {  
        int x = 0;  
  
        // O do-while garante que pelo menos  
        // uma vez o código seja executado  
  
        do {  
            System.out.println(x);  
            x++;  
        } while (x < 10);  
    }  
}
```

Primeiro foi definido o valor zero para variável x do tipo inteiro int. Na sequência, o comando do executa a repetição pela primeira vez e imprime o valor de x e incrementa (soma+1) o valor de x. Em seguida o while verifica se a variável x é menor que 10. Caso a variável x seja menor que 10 a repetição deve continuar. O programa encerra quando a instrução while testar uma condição falsa, ou seja x = 10.



FOR

- Através da instrução **for** é possível definir a quantidade de vezes que um determinado trecho de código deve ser executado pelo computador;
- Para utilizar a instrução **for** para realizar repetições você deve definir o valor inicial, o valor final e o valor de incremento.



Comando de Repetição

for

```
public class For1 {  
    public static void main(String[] args) {  
        for (int i = 0; i < 10; i++) {  
            System.out.print("item: " + i);  
        }  
    }  
}
```

O comando for tem 3 valores separados por ponto e virgula. O valor inicial da repetição foi definido através da variável `i` do tipo `int` com valor zero. O valor final da repetição é menor que 10 (`i < 10`), o programa deve repetir enquanto `i` tiver valor menor que 10. A instrução for deve incrementar em um (somar + 1) a variável `i` toda vez que ocorrer a repetição.

REFERÊNCIAS BIBLIOGRÁFICAS

ENCONTRE MAIS INFORMAÇÕES SOBRE ESTRUTURA DE CONTROLE JAVA

The Java Tutorials - Control Flow Statements

<https://docs.oracle.com/javase/tutorial/java/nutsandbolts/flow.html>

Core Java Guru - Control Flow Statements in Java

<http://www.corejavaguru.com/java/basic/controlstatements>

Sadao Massago - Programação Java - Controle de Fluxo

<https://www.dm.ufscar.br/~waldeck/curso/java/part26.html>

Obrigado

ESTRUTURAS DE CONTROLE

—
PROF. DOMINGOS BERNARDO